# LABORATORY OBSERVATION

**Course:** Full Stack Web Development
**Course Code:** 23CM4121
**Experiment No.:** 5
**Student Name:** Nimidithalli Nandu
**Roll No.:** A24126552100
**Date:** 20-09-2026

---

## 1. TITLE

**Develop a File Handling Application Using Node.js File System Module**

---

## 2. AIM

To develop a Node.js application that creates a file, writes content into the file, reads the file contents, appends additional content, and finally displays the updated file contents using the Node.js File System module.

---

## 3. OBJECTIVE

The objectives of this experiment are:
● To understand the basics of Node.js file handling.
● To use the built-in fs module in Node.js.
● To accept user input using the readline module.
● To create and write content into a file.
● To read data from a file.
● To append additional content to an existing file.
● To handle file operation errors using callback functions.
● To display the final contents of the modified file.

---

## 4. THEORY

Node.js provides several built-in modules that can be used to perform different operations without installing external packages.

The **File System (fs) module** is a built-in Node.js module used to work with files and directories. It provides functions for creating, reading, writing and modifying files.

The **Readline module** is used to read input from the user through the command-line interface. In this application, readline.createInterface() connects the standard input and output streams to allow interactive user input.

The application uses the following important functions:

- fs.writeFile() – creates a new file or overwrites an existing file with the specified content.
- fs.readFile() – reads the contents of a file.
- fs.appendFile() – adds additional content to an existing file.
- readline.createInterface() – creates an interface for reading user input.
- rl.question() – asks the user a question and receives the input through a callback function.
- rl.close() – closes the readline interface after completing the operations.

The application performs file operations in the following sequence:

**User Input → Create/Write File → Read File → Append Content → Read Final File → Display Output**

Callback functions are used to execute the next operation after the previous file operation is completed. Error handling is performed using the err parameter provided by the callback.

---

# 5. ALGORITHM / PROCEDURE

1. Create a new Node.js project.
2. Create a JavaScript file for the file handling application.
3. Import the built-in fs module using require().
4. Import the readline module using require().
5. Create a readline interface using readline.createInterface().
6. Connect process.stdin as the input stream.
7. Connect process.stdout as the output stream.
8. Ask the user to enter the file name.
9. Ask the user to enter the initial file content.
10. Use fs.writeFile() to create the file and write the entered content.
11. Check whether an error occurred during the write operation.
12. Display a success message if the file is created successfully.
13. Use fs.readFile() to read the contents of the created file.
14. Display the contents of the file.
15. Ask the user to enter additional content.
16. Use fs.appendFile() to add the additional content to the file.

17. Display a success message after appending the content.
18. Use fs.readFile() again to read the final contents of the file.
19. Display the final file contents.
20. Close the readline interface using rl.close().
21. Test the application using different file names and content.
22. Verify the expected and actual results.

---

# 6. PROGRAM

### Code.js

```
const fs = require("fs");

const readline = require("readline");

const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout
});

rl.question("Enter the file name: ", function (fileName) {

    rl.question("Enter the content: ", function (content) {

        // Create and write to the file

        fs.writeFile(fileName, content, function (err) {

            if (err) {

                console.log("Error creating file:", err);
                rl.close();
                return;

            }

            console.log("\nFile created successfully.");
```

```
// Read the file

fs.readFile(fileName, "utf8", function (err, data) {

    if (err) {

        console.log("Error reading file:", err);
        rl.close();
        return;

    }

    console.log("\nFile contents:");
    console.log(data);

    // Append additional content

    rl.question(
        "\nEnter additional content: ",
        function (extraContent) {

            fs.appendFile(
                fileName,
                "\n" + extraContent,
                function (err) {

                    if (err) {

                        console.log(
                            "Error appending file:",
                            err
                        );

                        rl.close();
                        return;
                    }

                    console.log(
                        "\nContent appended successfully."
```

```
                        );

                        // Read final contents

                        fs.readFile(
                            fileName,
                            "utf8",
                            function (err, finalData) {

                                if (err) {

                                    console.log(
                                        "Error reading final file:",
                                        err
                                    );

                                    rl.close();
                                    return;
                                }

                                console.log(
                                    "\nFinal file contents:"
                                );

                                console.log(finalData);

                                rl.close();
                            }
                        );
                    }
                );
            }
        );
    });
});
});
```

# 7. CODE EXPLANATION

| Code / Function | Explanation |
|---|---|
| `require("fs")` | Imports the built-in Node.js File System module. |
| `require("readline")` | Imports the Node.js Readline module. |
| `readline.createInterface()` | Creates an interface for accepting command-line input. |
| `process.stdin` | Represents the standard input stream from the user. |
| `process.stdout` | Represents the standard output stream. |
| `rl.question()` | Displays a question and accepts user input. |
| `fs.writeFile()` | Creates a file and writes the specified content into it. |
| `fs.readFile()` | Reads the contents of the specified file. |
| `"utf8"` | Specifies that the file should be read as UTF-8 text. |
| `fs.appendFile()` | Adds additional content to the existing file. |
| `err` | Contains error information if a file operation fails. |
| `data` | Contains the contents read from the file. |
| `finalData` | Contains the final contents after additional data is appended. |
| `console.log()` | Displays messages and file contents on the terminal. |

| | |
|---|---|
| `rl.close()` | Closes the readline interface. |
| `return` | Stops further execution when an error occurs. |

---

# 8. TEST CASES AND EXECUTION DATA

| Test Case | File Name | Initial Content | Additional Content | Expected Result |
|---|---|---|---|---|
| TC-01 | notes.txt | Hello Node.js | File Handling | File created and content appended successfully |
| TC-02 | student.txt | Student Record | Full Stack Development | File should contain both contents |
| TC-03 | data.txt | First Line | Second Line | Second content should be appended |

## TC-01 output:

```
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task5\Nodejs_File_Management> node code.js
 Enter the file name: notes.txt
 Enter the content: Hello Node.js

 File created successfully.

 File contents:
 Hello Node.js

 Enter additional content: File Handling

 Content appended successfully.

 Final file contents:
 Hello Node.js
 File Handling
```

## TC-02 Output:

```
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task5\Nodejs_File_Management> node code.js
Enter the file name: student.txt
Enter the content: Student Record

File created successfully.

File contents:
Student Record

Enter additional content: Full Stack Development

Content appended successfully.

Final file contents:
Student Record
Full Stack Development
```

## TC_03 Output:

```
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task5\Nodejs_File_Management> node code.js
Enter the file name: data.txt
Enter the content: First Line

File created successfully.

File contents:
First Line

Enter additional content: Second Line

Content appended successfully.

Final file contents:
First Line
Second Line
```

---

# 9. OUTPUT:

```
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task5\Nodejs_File_Management> node code.js
 Enter the file name: notes.txt
 Enter the content: Hello Node.js

 File created successfully.

 File contents:
 Hello Node.js

 Enter additional content: File Handling

 Content appended successfully.

 Final file contents:
 Hello Node.js
 File Handling
```

---

# 10. RESULT

Thus, a **File Handling Application** was successfully developed using **Node.js**, the **File System (fs) module**, and the **Readline module**. The application successfully created and wrote data to a file, read the file contents, appended additional content, and displayed the final updated contents. Error handling was also implemented for file operations.

---